LIVE CLASS NOTES

LangGraph & shipping

3 Nov 2026 · 19:55–22:02 · 126 min · Rahul Tiwari

Overview

This session focused on introducing LangGraph, a framework developed by the LangChain team specifically for building AI agents. The instructor walked participants through the conceptual foundations of LangGraph, its core components, and progressively built hands-on implementations ranging from simple sequential flows to conditional routing and ReAct-style agents with tool integration and short-term memory.

Key Concepts or Theories

- **Node**: Any Python function that performs a task within the agent workflow
- **Edge**: The connection between nodes that defines how data flows through the graph
- **State (Working Memory / Blackboard)**: A shared memory structure that all nodes read from and write to
- **Conditional Routing**: Logic that directs the agent to different nodes based on the current state
- **ReAct Architecture**: A pattern where the agent reasons and then acts, looping through tools until the goal is achieved
- **Short-Term Memory / Memory Saver**: An in-session memory mechanism using LangGraph's built-in checkpoint functionality
- **Thread ID**: A unique identifier used to maintain separate conversation contexts for different users
- **Pydantic**: A Python library used to enforce structured output formats from LLMs
- **LangChain**: A Python library providing common abstractions for connecting to various LLM providers and building workflows

Important Questions Raised

- How does a node know what the previous node performed? with the help of Working Memory or State
- Why use LangGraph instead of plain Python for building agents? it helps in setting up the flows and development efficiently
- How can LLM output be restricted to specific categories (e.g., positive or negative only)? with the help of setting up structured output
- How does short-term memory differ from long-term memory in agent systems? short term memory is lost once the application restarts
- How are different user conversations kept separate within the same agent? with the help of threads that help via the threadIds

---

Key Takeaways and Summary of Learning Objectives

- LangGraph is a low-level framework designed specifically for building agentic workflows, developed as a sub-library of the LangChain ecosystem
- The three core components of LangGraph are **Node**, **Edge**, and **State**, and understanding these is sufficient to build complex agent architectures
- Nodes are Python functions; edges define connections between nodes; state is the shared working memory updated by each node
- Conditional routing enables non-linear, dynamic agent flows where different nodes are triggered based on the current state
- LangChain's `ChatOpenAI`, `HumanMessage`, and structured output tools (Pydantic) integrate naturally with LangGraph
- Short-term memory can be implemented using LangGraph's `MemorySaver` and `thread_id` configuration
- Long-term memory requires external storage such as a vector database or relational database (e.g., PostgreSQL)
- Tools can be attached to agents using LangChain's `@tool` decorator and `ToolNode` from LangGraph's prebuilt module

---

Topic 1: LangGraph Background and Ecosystem

The LangChain team originally built a single Python library to simplify the development of GenAI applications, providing a unified interface for connecting to multiple LLM providers such as OpenAI, Anthropic, and Gemini. As the complexity of AI applications grew from simple prompt-response pipelines (circa 2022) to multi-step agentic workflows (circa 2024), the team split their ecosystem into specialized sub-libraries: **LangChain** for simple applications and deterministic workflows, **LangGraph** for building autonomous agents, and **LangSmith** for monitoring and observability.

LangGraph was introduced because managing agent orchestration — including loops, conditional branching, tool usage, and memory — under a single general-purpose library was not scalable. LangGraph provides a low-level, flexible framework that allows developers to define exactly how agents are connected and how data flows between them, making it suitable for production-grade agentic systems.

Relevant Q&A

Question: "Why not just use plain Python to build agents?"

Answer: Plain Python is valid and used in many production systems, but LangGraph provides reusable, plug-and-play components that significantly speed up agent development, especially for complex orchestration patterns.

Question: "How does LangGraph compare to other frameworks like CrewAI, Semantic Kernel, or Azure AI Foundry?"

Answer: Other frameworks tend to be more tightly coupled to their own ecosystems and are harder to customize. LangGraph is more flexible and lower-level, allowing developers to unfold and control the architecture in detail.

Topic 2: Core Components — Node, Edge, and State

LangGraph's architecture is built on three primitives. A **Node** is any Python function that performs a task; it always receives the current state as input and may update it. An **Edge** defines the connection between nodes, determining the order and conditions under which nodes are executed. The **State** (also called working memory or blackboard) is a shared data structure — defined using Python's `TypedDict` — that all nodes read from and write to, enabling nodes to operate independently without directly communicating with each other.

A `StateGraph` object is used to register nodes and edges. Every graph has a designated start point and end point. Nodes are added using `add_node()` and connected using `add_edge()` or `add_conditional_edges()`. Once the graph is compiled, it can be invoked with an initial state, and the framework handles the execution flow automatically.

Relevant Q&A

Question (Sharad Keshari): "Why is the routing function not shown as a node in the diagram?"

Answer: The routing function (`route_by_sentiment`) was used internally as a conditional edge function rather than an explicit node. It can be added as a node if desired, which would make the diagram clearer. The instructor confirmed this is a valid approach and walked through how the edges would need to be restructured accordingly.

Question (Shiva Shankar PP): "Is an edge like passing the output of one agent as the input to the next?"

Answer: Yes, edges define how data flows from one node to the next. They can be unidirectional or bidirectional depending on the use case.

Topic 3: Hands-On Implementation — Sequential and Conditional Flows

The instructor demonstrated two progressively complex implementations. The first was a **sequential flow** with two nodes (`process_input` and `add_prefix`), where each node reads from and writes to the shared state. The second was a **conditional routing flow** for sentiment analysis of product reviews. Three nodes were defined: `analyze_sentiment`, `generate_positive_response`, and `generate_negative_response`. The sentiment analysis node used a structured LLM output (via Pydantic's `Literal` type) to restrict responses to "positive" or "negative" only. A routing function then directed the flow to the appropriate response node based on the sentiment stored in the working memory.

Pydantic's `BaseModel` and `Literal` were used to enforce output structure, ensuring the LLM could not return values outside the defined categories. LangChain's `ChatOpenAI` class was used to initialize the GPT-4o-mini model, and the `with_structured_output()` method was applied to bind the Pydantic schema to the LLM.

Relevant Q&A

Question: "How do we decide what fields to include in the state?"

Answer: The state must be defined in advance based on the known requirements of the workflow. All information that agents need to share or that will be part of the final output should be included as fields in the state.

Question: "Why not have one node handle everything?"

Answer: Assigning all responsibilities to a single node reduces modularity and makes the agent less effective. Distributing responsibilities across specialized nodes mirrors good software design principles and improves agent performance.

Topic 4: ReAct Architecture and Tool Integration

The instructor introduced the **ReAct (Reason + Act)** pattern, where an agent iteratively decides whether to use a tool or respond directly. Using LangChain's `@tool` decorator, Python functions (e.g., `get_weather`, `calculator`) were converted into tools with descriptions that the LLM uses to decide which tool to invoke. The `ToolNode` from `langgraph.prebuilt` was used to register all tools as a single node in the graph.

The agent node was bound to the tools using LangChain's `bind_tools()` method. A conditional edge checked whether the agent's latest response was a tool call or a final answer, looping back to the tool node if needed or ending the process otherwise. This pattern enables agents to autonomously decide when and how to use tools without explicit programming of every decision path.

The instructor also noted the upcoming relevance of **Model Context Protocol (MCP)**, which aims to standardize tool definitions across frameworks, reducing the need to rewrite tool code when switching between LangGraph, CrewAI, or other frameworks.

Relevant Q&A

Question: "Can LangGraph work with image inputs instead of text?"

Answer: Yes. LangGraph is an orchestration layer and is agnostic to data type. Image data would need to be processed through an extraction layer first, and the resulting information can then flow through the LangGraph workflow.

Question: "Can we define a tool as a node?"

Answer: Yes, any Python function — including tool functions — can be registered as a node in LangGraph.

Topic 5: Short-Term Memory and Thread Management

To enable agents to remember previous interactions within a session, LangGraph provides a built-in `MemorySaver` class used as a checkpointer during graph compilation. When invoking the agent, a `config` dictionary containing a `thread_id` is passed to maintain separate conversation histories for different users or sessions. This allows the agent to answer follow-up questions such as "What was my last question?" correctly within the same session.

The instructor emphasized that `MemorySaver` is **short-term memory only** — it exists in Python's working memory and is lost when the notebook or session is closed. For production use cases, long-term memory must be implemented using external databases such as PostgreSQL or vector databases, which will be covered in future sessions.

Relevant Q&A

Question (Saida Begum): "Is MemorySaver storing data in our system?"

Answer: No. MemorySaver stores data in Python's in-memory variables only. Once the notebook session is closed, all data is lost. It is not written to the file system.

Question (Saida Begum): "How is this different from the chat history I see in ChatGPT?"

Answer: ChatGPT uses long-term memory, where conversations are persisted in a database. MemorySaver is a short-term, in-session equivalent. Long-term memory implementation using databases will be covered in future sessions.

Question (Raman Aggarwal): "Can we use Redis cache for short-term memory? And is KV cache the same as cache?"

Answer: Redis cache is a different mechanism from LangGraph's built-in MemorySaver. KV cache stands for key-value cache and is essentially the same concept as a general cache, just described in terms of its data structure. Both are related but serve different purposes in different contexts.

Actionable Next Steps / Assignments

this needs to be done

- Review and practice the code from **Session 1, Session 2, and Session 3** available in the GitHub repository
- Gain foundational understanding of **LangChain** by exploring the LangChain Learning folder in the course repository, as it is a prerequisite for working effectively with LangGraph
- Read the session-wise **notes available in the Documents folder** of the GitHub repository, particularly the sections on prompt engineering and agentic AI
- Explore the **BFL Chatbot** and **Inventor** projects in the repository for examples of long-term memory implementation using PostgreSQL

Supplemental Resources

- **GitHub Repository**: The instructor's full course repository containing session-wise notebooks, notes, code, and projects covering prompt engineering, RAG, NLP, and Agentic AI — including LangChain learning materials and advanced agent implementations https://github.com/rahul8879/coding_ninja_genai
- **LangGraph Documentation**: Official documentation recommended for reading about core concepts, available at the LangGraph official site
- **LangChain OpenAI Library**: `langchain-openai` pip package for connecting to OpenAI models via LangChain
- **LangGraph Prebuilt Module**: `langgraph.prebuilt` — contains `ToolNode` for converting Python tool functions into LangGraph nodes
- **Pydantic Library**: Used for defining structured output schemas to enforce LLM response formats

---